

SIMATS Engineering

SIMATS Online Programming Lab — Python Programming

Course Name	Python Programming
Course Code	CSA0804
Name	Monish.L
Register Number	1972260035
Topic	CO2-AT2-SCENARIO BASED ASSESSMENT
Question	Q4 — Scenario 4: Email Domain Frequency Analyser (20 Marks)

Scenario 4: Email Domain Frequency Analyser [20 Marks]

An organization maintains a large dataset of employee email addresses stored as strings with varying formats. The system must extract domain names from each email entry using string splitting. It should count how frequently each domain appears in the dataset. The most commonly used domain must be identified and reported. Control flow constructs must guide counting and comparison logic. Functions should be implemented for extraction and analysis processes. The program must handle invalid or malformed email addresses gracefully.

Task

Design a Python solution for comprehensive email domain frequency analysis, demonstrating the following key skills:

- String splitting using `split('@')`
- Domain extraction and storage
- Frequency counting using loops and conditionals
- Identifying maximum frequency
- Handling malformed emails using conditionals

Approach / Design

- **Validation (is_valid_email):** checks that an email has exactly one '@', a non-empty local part, and a domain part containing a '.', before any extraction is attempted.
- **Domain extraction (extract_domain):** uses split('@') to split the email into local and domain parts and returns the lowercased domain.
- **Batch extraction (extract_all_domains):** loops through every email in the dataset, routing valid entries to domain extraction and gracefully collecting malformed entries separately instead of raising an error.
- **Frequency counting (count_domain_frequency):** loops over the extracted domains and uses conditional dictionary updates to tally occurrences of each domain.
- **Most-common lookup (find_most_common_domain):** loops through the frequency dictionary and uses comparison logic to track the highest-count domain explicitly, rather than relying on a built-in shortcut.
- **Full analysis (analyze_email_dataset):** orchestrates extraction, counting, and top-domain lookup into one summary dictionary.
- **Reporting (print_report):** prints valid/invalid counts, the skipped malformed entries, per-domain frequency, and the most common domain.

Python Program

```
"""
C02-AT2 - Scenario Based Assessment
Scenario 4: Email Domain Frequency Analyser
-----
An organization maintains a dataset of employee email addresses
in varying formats. This program:
    1. Extracts the domain from each email using split('@')
    2. Stores extracted domains for analysis
    3. Counts how frequently each domain appears using loops
       and conditionals
    4. Identifies the most commonly used domain
    5. Gracefully handles invalid/malformed email addresses
    6. Is fully modularized using functions for extraction
       and analysis
"""

# -----
# 1. Sample raw email dataset (includes malformed entries)
# -----
RAW_EMAILS = [
    "alice@company.com",
    "bob@company.com",
    "carol@partner.org",
    "dave@company.com",
    "eve@vendor.net",
    "frank@partner.org",
    "not-an-email",           # malformed: no '@'
    "grace@@company.com",    # malformed: multiple '@'
    "heidi@",                # malformed: no domain
    "@company.com",          # malformed: no local part
    "ivan@company.com",
    "judy@vendor.net",
]

# -----
```

```

# 2. Validation function
# -----
def is_valid_email(email):
    """
    Performs a basic structural check on an email address:
    exactly one '@', a non-empty local part, and a non-empty
    domain part containing at least one '.'.
    """
    if email.count("@") != 1:
        return False

    local_part, domain_part = email.split("@")

    if len(local_part) == 0 or len(domain_part) == 0:
        return False

    if "." not in domain_part:
        return False

    return True

# -----
# 3. Domain extraction function
# -----
def extract_domain(email):
    """
    Uses split('@') to extract the domain portion of a valid
    email address.
    """
    return email.split("@")[1].strip().lower()

# -----
# 4. Batch extraction function (handles malformed entries)
# -----
def extract_all_domains(emails):
    """
    Loops through every email in the dataset. Valid emails have
    their domain extracted and stored; malformed emails are
    gracefully skipped and logged separately instead of crashing
    the program.
    """
    valid_domains = []
    invalid_emails = []

    for email in emails:
        if is_valid_email(email):
            domain = extract_domain(email)
            valid_domains.append(domain)
        else:
            invalid_emails.append(email)

    return valid_domains, invalid_emails

# -----
# 5. Frequency counting function
# -----
def count_domain_frequency(domains):
    """
    Counts how frequently each domain appears using a loop and
    conditional dictionary updates.
    """
    frequency = {}
    for domain in domains:
        if domain in frequency:
            frequency[domain] += 1
        else:

```

```

        frequency[domain] = 1
    return frequency

# -----
# 6. Most-common-domain identification function
# -----
def find_most_common_domain(frequency):
    """
    Identifies the domain with the highest frequency using a
    loop and comparison logic (no built-in max() shortcut,
    to explicitly demonstrate control flow).
    """
    if not frequency:
        return None, 0

    top_domain = None
    top_count = 0

    for domain, count in frequency.items():
        if count > top_count:
            top_domain = domain
            top_count = count

    return top_domain, top_count

# -----
# 7. Full analysis function
# -----
def analyze_email_dataset(emails):
    """
    Orchestrates extraction, frequency counting, and identification
    of the most common domain, returning a summary dictionary.
    """
    valid_domains, invalid_emails = extract_all_domains(emails)
    frequency = count_domain_frequency(valid_domains)
    top_domain, top_count = find_most_common_domain(frequency)

    return {
        "total_emails": len(emails),
        "valid_count": len(valid_domains),
        "invalid_count": len(invalid_emails),
        "invalid_emails": invalid_emails,
        "frequency": frequency,
        "top_domain": top_domain,
        "top_count": top_count,
    }

# -----
# 8. Report/display function
# -----
def print_report(summary):
    print("=" * 70)
    print("EMAIL DOMAIN FREQUENCY ANALYSIS REPORT")
    print("=" * 70)

    print(f"\nTotal email entries      : {summary['total_emails']}")
    print(f"Valid email entries          : {summary['valid_count']}")
    print(f"Invalid/malformed entries    : {summary['invalid_count']}")

    if summary["invalid_emails"]:
        print("\nMalformed entries skipped:")
        for entry in summary["invalid_emails"]:
            print(f"  - {entry!r}")

    print("\nDomain frequency counts:")
    for domain, count in summary["frequency"].items():

```

```

        print(f"  {domain:<20}: {count}")

    if summary["top_domain"]:
        print(
            f"\nMost commonly used domain: {summary['top_domain']} "
            f"({summary['top_count']} occurrences)"
        )
    else:
        print("\nNo valid domains found to analyze.")

# -----
# 9. Main driver
# -----
def main():
    summary = analyze_email_dataset(RAW_EMAILS)
    print_report(summary)

if __name__ == "__main__":
    main()

```

Sample Output

```
=====
EMAIL DOMAIN FREQUENCY ANALYSIS REPORT
=====

Total email entries      : 12
Valid email entries      : 8
Invalid/malformed entries : 4

Malformed entries skipped:
- 'not-an-email'
- 'grace@@company.com'
- 'heidi@'
- '@company.com'

Domain frequency counts:
company.com      : 4
partner.org      : 2
vendor.net       : 2

Most commonly used domain: company.com (4 occurrences)
```

Conclusion

The program extracts domains from a mixed dataset of email addresses using `split('@')`, validating each entry first so malformed addresses (missing '@', multiple '@' symbols, empty local or domain parts) are gracefully skipped rather than crashing the program. Frequency counting and the most-common-domain lookup both use explicit loops and comparison logic rather than relying on library shortcuts, and each responsibility is isolated in its own function, making the solution easy to extend to larger, real-world email datasets.